МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 2
«Розроблення підсистеми розмежування доступу»
з дисципліни «Безпека програмного забезпечення»

Варіант 14

Виконав:
здобувач групи ІП-34
Скачков І. О.

Перевірив:
Курченко О. А.

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи — набути практичних навичок побудови та перевірки диспетчера доступу (reference monitor), що реалізує задану модель розмежування доступу, веде журнали своїх рішень і виявляє аномалії в поведінці суб'єктів.

Завдання роботи:

1) реалізувати диспетчер доступу за моделлю варіанта як підклас базового класу Policy;
2) наповнити політику суб'єктами, об'єктами й правами в кількостях варіанта та застосувати додаткове правило;
3) перевірити три властивості еталонного монітора: повне посередництво, захищеність і верифіковність;
4) реалізувати реєстраційний та операційний журнали й детектор двох аномалій варіанта;
5) написати власні модульні тести за зразком tests/test_example.py;
6) продемонструвати запити «хто має доступ до об'єкта» та «до чого має доступ суб'єкт».

Параметри варіанта отримано командою `python -m common.variants 14 --lab 2`:

```
  Варіант 14

    LAB2
      model: RBAC з ієрархією ролей
      extra_rule: тимчасові права з терміном дії
      anomalies:
        - сплеск відмов у доступі від одного суб'єкта
        - використання прав, не задіяних раніше
      subjects: 10
      objects: 23
```

Таблиця 1.1 – Індивідуальне завдання (варіант 14)

Параметр	Значення
Модель	M3 — RBAC з ієрархією ролей
Додаткове правило	D2 — тимчасові права з терміном дії
Аномалії	A1 — сплеск відмов у доступі від одного суб'єкта; A5 — використання прав, не задіяних раніше
Суб'єктів / об'єктів	10 / 23

2 ТЕОРЕТИЧНЕ ОБҐРУНТУВАННЯ ПРИЙНЯТИХ РІШЕНЬ

2.1 Модель RBAC і формула рішення

Політику побудовано для системи електронного документообігу кафедри з ЛР1. У RBAC права призначаються не суб'єктам, а ролям: UA — «суб'єкт → роль», PA — «роль → права на об'єкт», RH — ієрархія ролей, у якій старша роль успадковує всі права молодших. Для кафедри це природно: права визначаються посадою (викладач, секретар, завідувач), а зміна складу працівників зводиться до зміни UA без перегляду прав на 23 документи.

Формулу (3.1) практикуму адаптовано до RBAC: perm(s, o) — об'єднання прав на o усіх ролей із транзитивного замикання RH*(r) для кожної ролі r, призначеної s; заборонних записів у варіанті немає, тож друга умова тотожно істинна. Ієрархію збережено як орієнтований ациклічний граф (у ролі head дві молодші — lecturer і secretary, обидві успадковують employee); додавання ребра, що створює цикл, відхиляється.

2.2 Тимчасові права (D2)

Термін дії задається інтервалом [valid_from, valid_until) і може бути в обох видах призначень: в UA (тимчасова роль — в. о. завідувача) і в PA (тимчасове право ролі — редагування відомостей лише під час сесії). Інтервал напіввідкритий: у момент valid_until право вже не діє, тож межа однозначна. Час береться з годинника, переданого в політику, а не з глобального time.time(): це робить рішення відтворюваними в тестах і дає змогу записувати в журнал той самий час, за яким ухвалено рішення.

Відмову через неактивне тимчасове право відрізнено від відмови за відсутності права: у першому випадку причина містить «термін дії сплив …» або «діє лише з …». Це потрібно і для розслідування за журналом, і для детектора A1 (див. розділ 5).

2.3 Журнали та детектори

Реєстраційний журнал (AuditLog каркаса) фіксує кожне рішення монітора — і дозвіл, і відмову. Операційний журнал (OperationLog) фіксує лише виконані зміни стану: зміни політики (assign, permit, revoke, add_inheritance) і виконані операції WRITE / DELETE над документами. Пороги детекторів передаються параметрами, як у FanInDetector навчального прикладу, щоб калібрувати чутливість без зміни коду.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структура та програмні засоби

```
SoS/lab02_access_control/
  accesscontrol/monitor.py   каркас курсу (без змін)
  examples/, tests/test_example.py   навчальний приклад BLP (без змін)
  mypolicy.py      VariantPolicy (RBAC + D2), журнали, DocumentService
  population.py    10 суб'єктів, 23 об'єкти, ролі та призначення
  mydetector.py    детектори A1 і A5
  demo.py          демонстрація всіх кроків
  tests/test_variant.py   власні тести (29)
```

Використано Python 3.12 і pytest 8.3.3 (requirements.txt каркаса). Каркас не змінювався: власний код лише успадковує Policy, AuditLog і AnomalyDetector.

3.2 Структурна схема підсистеми

Схему наведено на рис. 3.1. Суб’єкти звертаються лише до DocumentService, який не має посилання на політику: кожна операція спершу викликає ReferenceMonitor.enforce, монітор викликає VariantPolicy.evaluate і записує рішення в реєстраційний журнал. Довірена обчислювальна база — монітор, метод evaluate з обходом ієрархії та годинник.

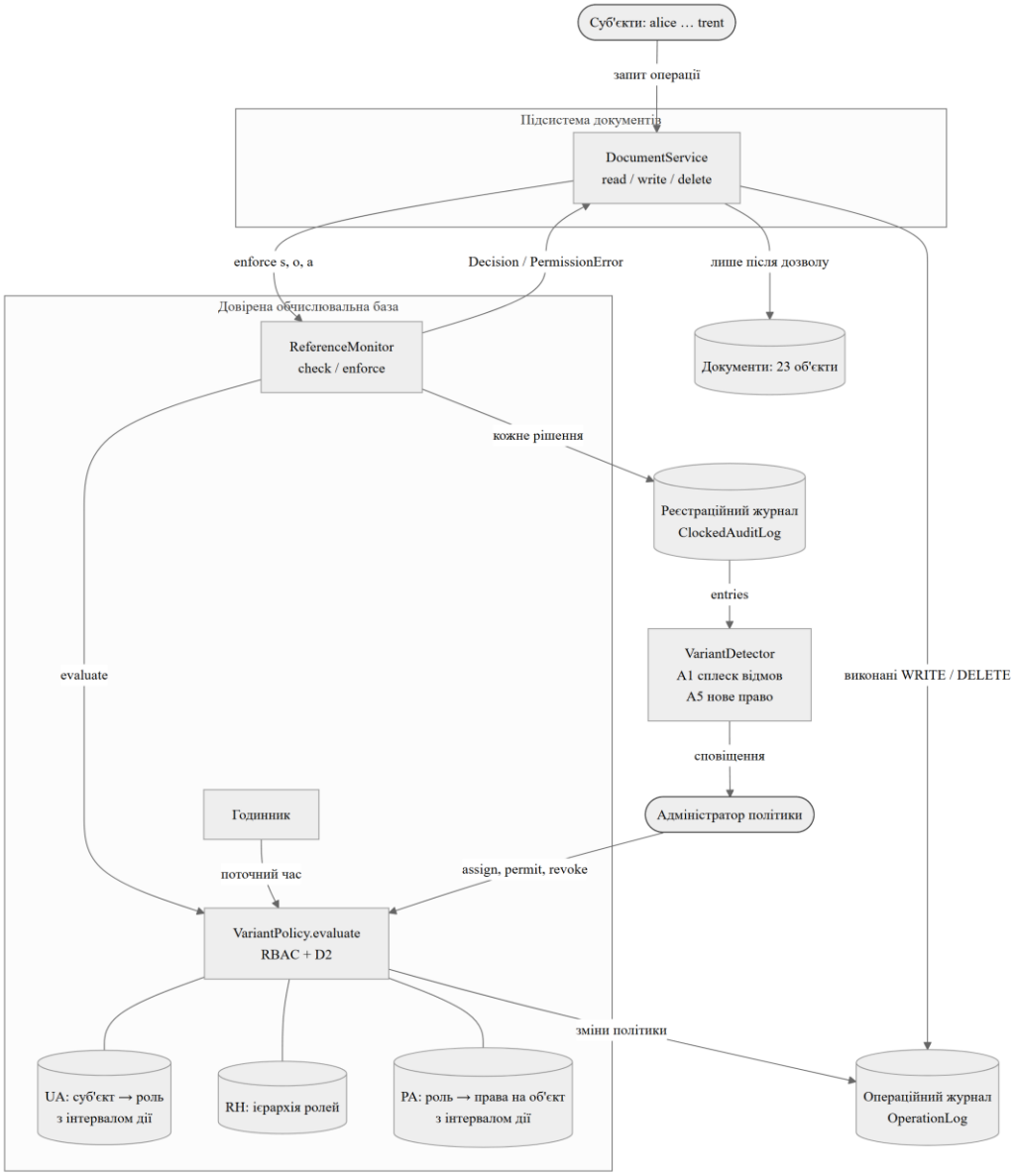


Рисунок 3.1 – Структурна схема підсистеми розмежування доступу

3.3 Наповнення політики

Політика містить 7 ролей (табл. 3.1) і 10 суб’єктів (табл. 3.2); об’єкти — 5 наказів order01–05, 6 відомостей grades01–06, 4 протоколи protocol01–04, 4 індивідуальні плани plan01–04 і 4 кадрові документи hr01–04. Усі імена умовні.

Таблиця 3.1 – Ролі, ієрархія та призначені права (PA)

Роль	Успадковує	Власні права	Тимчасові права
employee	—	READ order*, protocol*	—
lecturer	employee	READ|WRITE plan*, READ grades*	WRITE grades* до 16:00 26.10 (сесія)
secretary	employee	WRITE order*, protocol*	—
hr	employee	READ|WRITE|DELETE hr*	—

Роль	Успадковує	Власні права	Тимчасові права
head	lecturer, secretary	DELETE order*, READ hr*, WRITE grades*	—
student	—	READ order01	—
auditor	—	READ усіх 23 об’єктів	—

Таблиця 3.2 – Призначення ролей (UA)

Суб’єкт	Роль	Інтервал дії
alice	head	безстроково
bob	secretary	безстроково
carol, erin	lecturer	безстроково
dave	lecturer; head (в. о. завідувача)	lecturer — безстроково; head — 09:00–12:00 26.10.2026
frank	hr	безстроково
grace, heidi, mallory	student	безстроково
trent	auditor	безстроково

3.4 Ключові фрагменти коду

Метод evaluate (mypolicy.py) обходить призначені ролі суб’єкта та їх замикання за ієрархією й збирає два набори прав: чинні зараз (effective) і ті, що існують, але поза інтервалом дії (inactive). Дозвіл — лише якщо всі біти дії є в effective; інакше — відмова з поясненням:

```
def evaluate(self, subject: str, obj: str, action: Permission) -> Decision:
    now = self._clock()
    effective = Permission.NONE   # права, чинні зараз
    inactive = Permission.NONE    # права поза інтервалом дії (D2)
    why_inactive, sources = "", []
    for a in self._ua.get(subject, []):
        for role in sorted(self._closure(a.role)):
            for g in self._pa.get((role, obj), []):
                if not g.perm & action:
                    continue
                if a.interval.contains(now) and g.interval.contains(now):
                    effective |= g.perm
                    sources.append(_via(a.role, role))
                else:
                    inactive |= g.perm
                    why_inactive = why_inactive or _why(a, role, obj, g, now)
    if action & effective == action:
        via = ", ".join(dict.fromkeys(sources))
        return Decision(True, "надано через роль " + via)
    if action & (effective | inactive) == action:
        return Decision(False, "тимчасове право неактивне — " + why_inactive)
    return Decision(False, "явного дозволу немає (safe default)")
```

Транзитивне замикання ієрархії обчислюється обходом у глибину з множиною відвіданих вершин, тому спільна молодша роль (employee для head) враховується один раз:

```
def _closure(self, role: str) -> set[str]:
    """Роль разом з усіма молодшими (транзитивне замикання RH)."""
    seen, stack = set(), [role]
    while stack:
        r = stack.pop()
        if r not in seen:
            seen.add(r)
            stack.extend(self._juniors.get(r, ()))
    return seen
```

Детектор A1 (mydetector.py) для кожного суб'єкта сортує моменти відмов і ковзним вікном двома вказівниками знаходить найбільшу кількість відмов за window секунд:

```
def scan(self, log: AuditLog) -> list[str]:
    denials: dict[str, list[float]] = collections.defaultdict(list)
    for r in log.entries():
        if not r["allowed"]:
            denials[r["subject"]].append(r["ts"])
    alerts = []
    for subject, times in sorted(denials.items()):
        times.sort()
        best, best_start, left = 0, 0.0, 0
        for right, t in enumerate(times):          # ковзне вікно двома вказівниками
            while t - times[left] > self.window:
                left += 1
            if right - left + 1 > best:
                best, best_start = right - left + 1, times[left]
        if best >= self.threshold:
            alerts.append(f"A1: {subject} — {best} відмов за {self.window:.0f} с "
                          f"(від {fmt_ts(best_start)})")
    return alerts
```

Детектор A5 проходить журнал у порядку часу й накопичує профіль використаних прав суб'єкта; дозволене звернення після кінця базового періоду з ключем, якого немає в профілі, породжує сповіщення:

```
def scan(self, log: AuditLog) -> list[str]:
    used: dict[str, set[tuple[str, ...]]] = collections.defaultdict(set)
    alerts = []
    for r in sorted(log.entries(), key=lambda e: e["ts"]):
        if not r["allowed"]:
            continue
        key = self._key(r)
        if r["ts"] >= self.baseline_until and key not in used[r["subject"]]:
            alerts.append(f"A5: {r['subject']} уперше використав {'/'.join(key)} "
                          f"({r['object']}, {fmt_ts(r['ts'])})")
        used[r["subject"]].add(key)
    return alerts
```

3.5 Три властивості диспетчера

Повне посередництво. Єдиний шлях до документів — DocumentService, а кожен його метод першим рядком викликає monitor.enforce; виконання й запис в операційний журнал відбуваються лише після дозволу. Тест test_complete_mediation_every_call_logged перевіряє, що кількість записів журналу

дорівнює кількості звернень, а test_enforce_raises_and_denied_operation_is_not_executed — що відхилена операція не змінює стан.

Захищеність. DocumentService отримує лише монітор, а не політику (test_service_has_no_reference_to_policy); структури UA, PA, RH приховано в атрибутах із підкресленням, зміни політики виконуються лише методами адміністрування й потрапляють в операційний журнал. Обмеження: Python не забороняє доступ до приватних атрибутів, тому в реальній системі політика мала б працювати в окремому процесі.

Верифіковність. Довірена частина — evaluate (22 рядки) із двома допоміжними функціями _via і _why та _closure (8 рядків); вони покриті 29 тестами, зокрема перевіркою повної узгодженості запитів «хто» і «до чого» на всіх парах.

4 РЕЗУЛЬТАТИ

4.1 Очікувані й фактичні рішення

Таблиця 4.1 отримана скриптом demo.py: для 17 показових пар наведено очікуване рішення, фактичний результат evaluate і результат тієї самої політики без правила D2 (базова RBAC). Фактичні рішення збіглися з очікуваними в 17 з 17 випадків; правило D2 змінило результат у двох рядках (позначено «D2»).

Таблиця 4.1 – Очікувані й фактичні рішення evaluate

Час	Суб'єкт	Дія	Об'єкт	Очік.	Факт.	База	Причина рішення
10:00	alice	READ	hr01	дозвіл	дозвіл	дозвіл	надано через роль head
10:00	alice	DELETE	order02	дозвіл	дозвіл	дозвіл	надано через роль head
10:00	bob	WRITE	order03	дозвіл	дозвіл	дозвіл	надано через роль secretary
10:00	bob	READ	hr01	відмова	відмова	відмова	явного дозволу немає (safe default)
10:00	carol	READ	protocol01	дозвіл	дозвіл	дозвіл	надано через роль lecturer ← employee
10:00	carol	WRITE	grades01	дозвіл	дозвіл	дозвіл	надано через роль lecturer
17:00	carol	WRITE	grades01	відмова	відмова	дозвіл (D2)	тимчасове право неактивне — право lecturer на grades01: термін дії сплив 2026-10-26 16:00
10:00	dave	DELETE	order02	дозвіл	дозвіл	дозвіл	надано через роль head
13:00	dave	DELETE	order02	відмова	відмова	дозвіл (D2)	тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00
13:00	dave	WRITE	plan01	дозвіл	дозвіл	дозвіл	надано через роль lecturer
10:00	frank	DELETE	hr02	дозвіл	дозвіл	дозвіл	надано через роль hr
10:00	grace	READ	order01	дозвіл	дозвіл	дозвіл	надано через роль student

Час	Суб'єкт	Дія	Об'єкт	Очік.	Факт.	База	Причина рішення
10:00	grace	READ	order02	відмова	відмова	відмова	явного дозволу немає (safe default)
10:00	mallory	READ	grades02	відмова	відмова	відмова	явного дозволу немає (safe default)
10:00	trent	READ	hr03	дозвіл	дозвіл	дозвіл	надано через роль auditor
10:00	trent	WRITE	hr03	відмова	відмова	відмова	явного дозволу немає (safe default)
10:00	eve	READ	order01	відмова	відмова	відмова	явного дозволу немає (safe default)

4.2 Журнали

Серія з 29 звернень через DocumentService (робочий день 26.10.2026, 09:05–17:00) дала 29 записів реєстраційного журналу, з них 10 відмов. Операційний журнал містить 100 записів про зміни політики під час наповнення і 8 записів про виконані зміни документів; читання й відхилені операції в нього не потрапили. Фрагмент реєстраційного журналу після базового періоду — у табл. 4.2, фрагмент у форматі AuditLog.dump:

```
{"ts": 1793001900.0,
 "subject": "carol",
 "object": "grades01",
 "action": "WRITE",
 "allowed": true,
 "reason": "надано через роль lecturer"}
{"ts": 1793003400.0,
 "subject": "mallory",
 "object": "grades01",
 "action": "READ",
 "allowed": false,
 "reason": "явного дозволу немає (safe default)"}
```

Таблиця 4.2 – Реєстраційний журнал (записи після 10:00)

Час	Суб'єкт	Дія	Об'єкт	Рішення	Причина
10:05:00	carol	WRITE	grades01	дозвіл	надано через роль lecturer
10:30:00	mallory	READ	grades01	відмова	явного дозволу немає (safe default)
10:30:08	mallory	READ	grades02	відмова	явного дозволу немає (safe default)
10:30:16	mallory	READ	grades03	відмова	явного дозволу немає (safe default)
10:30:24	mallory	READ	grades04	відмова	явного дозволу немає (safe default)
10:30:32	mallory	READ	grades05	відмова	явного дозволу немає (safe default)
10:30:40	mallory	READ	grades06	відмова	явного дозволу немає (safe default)
11:00:00	dave	DELETE	order04	дозвіл	надано через роль head
12:30:00	dave	DELETE	order05	відмова	тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00
12:31:00	dave	DELETE	order05	відмова	тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00
15:00:00	erin	WRITE	grades02	дозвіл	надано через роль lecturer
16:30:00	erin	WRITE	grades02	відмова	тимчасове право неактивне — право lecturer на grades02: термін дії сплив 2026-10-26 16:00
17:00:00	trent	READ	order03	дозвіл	надано через роль auditor

Таблиця 4.3 – Операційний журнал (зміни документів)

Час	Суб'єкт	Операція	Об'єкт
09:10	alice	write	grades01
09:13	bob	write	order02
09:20	bob	write	protocol01
09:26	carol	write	plan01
09:46	frank	write	hr02
10:05	carol	write	grades01
11:00	dave	delete	order04
15:00	erin	write	grades02

4.3 Спрацювання детекторів

На тому самому журналі VariantDetector(baseline_until = 10:00, threshold = 5, window = 60 с) видав три сповіщення:

```
A1: mallory — 6 відмов за 60 с (від 2026-10-26 10:30)
A5: dave уперше використав DELETE (order04, 2026-10-26 11:00)
A5: erin уперше використав WRITE (grades02, 2026-10-26 15:00)
```

На «чистому» журналі (16 записів звичайної роботи без перебору й нових прав) обидва детектори повертають порожній список. Із granularity="object" детектор A5 видає 4 сповіщення: додатково carol (перший WRITE саме відомостей) і trent (перший READ наказу).

4.4 Запити «хто має доступ» і «до чого має доступ»

Таблиця 4.4 – Результати запитів до та після закінчення ролі в. о. завідувача

Час	Хто видаляє order05	Хто пише grades03	Що видаляє dave	Ролі dave
10:00	alice, dave	alice, carol, dave, erin	order01, order02, order03, order04, order05	employee, head, lecturer, secretary
13:00	alice	alice, carol, dave, erin	—	employee, lecturer

Запит «до чого має доступ» реалізовано двічі: повним обходом об'єктів (objects_accessible_by) і через рольовий рівень (accessible_objects); результати збіглися для всіх суб'єктів. Узгодженість «$s \in$ хто(o) $\Leftrightarrow$ o $\in$ що(s)» перевірено для 2070 пар (10 суб'єктів × 23 об'єкти × 3 дії × 3 моменти часу) — порушень немає.

4.5 Спливання тимчасового права під час серії звернень

Таблиця 4.5 – Серія звернень dave до order05 (DELETE)

Час	Рішення	Причина	Хто видаляє order05
11:58	дозвіл	надано через роль head	alice, dave
11:59	дозвіл	надано через роль head	alice, dave
12:00	відмова	тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00	alice
12:01	відмова	тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00	alice

4.6 Тестування

Власні тести tests/test_variant.py (29 функцій test_*) охоплюють усі кроки 2–7: модель і успадкування (7), безпечні значення за замовчуванням (3), правило D2 (5), монітор і журнали (6), аномалії (6), запити (2). Результат запуску `PYTHONPATH=. pytest tests/ -q` разом із тестами прикладу — 39 passed; повний звіт pytest — у додатку Б.

```
29 passed in 0.19s
```

5 АНАЛІЗ РЕЗУЛЬТАТІВ

5.1 Складність запитів у RBAC

У RBAC обидва запити проходять через проміжний рівень ролей, але не симетрично щодо збережених структур. Запит «до чого має доступ s» природний: UA(s) → замикання ролей → PA цих ролей; accessible_objects переглядає лише записи PA активних ролей і не залежить від загальної кількості об’єктів. Запит «хто має доступ до o» потребує зворотного обходу: PA(o) → ролі → усі старші за ними ролі → суб’єкти, призначені на них. Індекси збережено в прямому напрямку (суб’єкт → ролі, старша → молодші), тому subjects_with_access каркаса виконує повний обхід 10 суб’єктів з evaluate для кожного. Для ACL співвідношення протилежне. Окремо D2 додає вимір часу: відповіді на обидва запити коректні лише для конкретного моменту (табл. 4.4).

5.2 Журнал і повне посередництво

Серія з 29 звернень дала рівно 29 записів реєстраційного журналу (і ще 4 записи в серії табл. 4.5 — 4 виклики). Жоден виклик не «загубився» і жоден запис не з'явився без виклику, що підтверджує повне посередництво: DocumentService не має іншого шляху до документів, ніж через enforce. Операційний журнал при цьому містить лише 8 змін документів — дозволені WRITE і DELETE; відмови й читання стану не змінюють.

5.3 Вплив правила D2

Порівняно з базовою RBAC правило D2 змінило 2 з 17 рішень. Пара dave — order02 (DELETE) о 13:00: базова модель дозволяє через роль head, але призначення dave → head діяло до 12:00, тому — відмова з причиною «призначення ролі head: термін дії сплив». Пара carol — grades01 (WRITE) о 17:00: право lecturer на редагування відомостей обмежене сесією до 16:00. Водночас dave о 13:00 зберігає безстрокову роль lecturer (WRITE plan01 дозволено), а alice — безстрокове WRITE на відомості через head.

5.4 Умови спрацювання детекторів і вибір порогів

A1 (5 відмов за 60 с) спрацював на mallory: 6 відмов за 40 с під час перебору grades01–06. Не спрацював на dave (2 відмови о 12:30–12:31 після спливання ролі) і на grace (одна відмова о 09:57). Тест test_a1_slow_denials_not_detected показує, що 5 відмов із проміжком 2 хв поріг не перетинають. Поріг 5 обрано тому, що легітимний користувач у сценарії отримує щонайбільше 2 відмови поспіль (повторна спроба після спливання права), а перебір документів дає серію за секунди; вікно 60 с відповідає темпу ручної роботи.

A5 (базовий період до 10:00, ключ «суб'єкт → дія») спрацював на dave — перший DELETE (order04 о 11:00) і на erin — перший WRITE (grades02 о 15:00). Не спрацював на carol (WRITE відомості о 10:05), бо WRITE вона вже використовувала для plan01 у базовому періоді, і на trent (READ уже в профілі).

Базовий період — перша година робочого дня — обраний для демонстрації; у реальній системі профіль слід навчати щонайменше тиждень.

5.5 Хибні спрацювання

Хибнопозитивні. Сповіщення A5 для erin — легітимна робота з відомостями під час сесії: профіль за годину просто не встиг побачити цю дію. Сповіщення для dave формально коректне (нове право справді вперше використане), але дія легітимна — в. о. завідувача. Для A1 хибнопозитивне спрацювання дасть послідовність: о 12:30 dave п’ять разів за хвилину повторює DELETE order05 після спливання ролі. Зменшити такі спрацювання можна, не рахуючи в A1 відмови з причиною «тимчасове право неактивне» або піднявши поріг для них.

Хибнонегативні. Порушник, що перебирає відомості по 4 за хвилину, A1 не помітить; той, хто заволодів обліковим записом carol і змінює відомості, A5 з ключем «дія» не помітить, бо WRITE уже в профілі. Ключ granularity="object" закриває другий випадок ціною більшої кількості сповіщень (4 замість 2 для A5 у сценарії).

6 ВИСНОВКИ

1) Реалізовано диспетчер доступу за моделлю RBAC з ієрархією ролей (7 ролей, ієрархія без циклів, перевірка циклу при додаванні) для 10 суб’єктів і 23 об’єктів; на 17 показових парах фактичні рішення evaluate збіглися з очікуваними в усіх випадках.
2) Правило D2 реалізовано для призначень UA і PA з напіввідкритим інтервалом дії; воно змінило 2 з 17 рішень порівняно з базовою RBAC, а відмова через сплив права має окрему причину в журналі.
3) Повне посередництво підтверджено журналом: 29 звернень — 29 записів реєстраційного журналу; операційний журнал фіксує лише 8 виконаних змін документів і 100 змін політики.
4) Детектори A1 (5 відмов / 60 с) і A5 (перше використання дії після базового періоду) на сценарії дали 1 і 2 сповіщення відповідно, на «чистому»

журналі — жодного; одне сповіщення A5 (erin) є хибнопозитивним через короткий базовий період.

5) Запити «хто має доступ» і «до чого має доступ» узгоджені на 2070 парах; 29 власних тестів і 10 тестів прикладу проходять (39 passed).

7 ВІДОМОСТІ ПРО ВИКОРИСТАННЯ ГЕНЕРАТИВНИХ ІНСТРУМЕНТІВ

Під час виконання роботи як допоміжний засіб використано генеративну модель Claude (Anthropic): для підготовки чернетки коду mypolicy.py, mydetector.py, тестів і демонстраційного сценарію, а також оформлення звіту. Результати перевірено запуском тестів і демонстрації; рішення здобувач перевірив і може пояснити під час захисту.

ПЕРЕЛІК ПОСИЛАНЬ

1) Saltzer J. H., Schroeder M. D. The Protection of Information in Computer Systems // Proceedings of the IEEE. – 1975. – Vol. 63, № 9. – P. 1278–1308.
2) Sandhu R. S., Coyne E. J., Feinstein H. L., Youman C. E. Role-Based Access Control Models // IEEE Computer. – 1996. – Vol. 29, № 2. – P. 38–47.
3) Bertino E., Bonatti P. A., Ferrari E. TRBAC: A Temporal Role-Based Access Control Model // ACM Transactions on Information and System Security. – 2001. – Vol. 4, № 3. – P. 191–233.
4) Anderson R. Security Engineering. – 3rd ed. – Indianapolis : Wiley, 2020. – 1232 p.
5) Bishop M. Computer Security: Art and Science. – 2nd ed. – Boston : Addison-Wesley, 2018.
6) Безпека програмного забезпечення. Частина 2. Лабораторний практикум / В. В. Соколовський. – Київ : КПІ ім. Ігоря Сікорського, 2026.

ДОДАТОК А

Репозиторій і артефакти роботи

Репозиторій: https://github.com/ip34is/7-sem (папка SoS)

Коміт, що відповідає зданій роботі:

`7766af32a6e667af3fa1700c397b65fabb344a12`

Стан роботи на цьому коміті: https://github.com/ip34is/7-sem/tree/7766af32a6e667af3fa1700c397b65fabb344a12/SoS/lab02_access_control

Артефакти: lab02_access_control/mypolicy.py, mydetector.py, population.py, demo.py, tests/test_variant.py; у каталозі звіт_лр2 — scheme.mmd / scheme.png (структурна схема), demo_output.txt, pytest_output.txt, audit_log.jsonl, operation_log.jsonl.

Відтворення результатів (з папки SoS/lab02_access_control):

```
pip install -r requirements.txt
PYTHONPATH=. pytest tests/ -q          # 39 passed
python demo.py --save ../звіт_лр2
```

ДОДАТОК Б

Звіт pytest для tests/test_variant.py

```
test_population_matches_variant PASSED
test_direct_role_permission PASSED
test_permission_inherited_from_junior_role PASSED
test_senior_role_inherits_both_branches PASSED
test_junior_does_not_get_senior_rights PASSED
test_combined_action_needs_all_bits PASSED
test_cycle_in_hierarchy_rejected PASSED
test_safe_default_unknown_subject PASSED
test_safe_default_unknown_object PASSED
test_empty_policy_denies_everything PASSED
test_temporary_role_assignment_expires PASSED
test_temporary_role_not_yet_active PASSED
test_expired_role_keeps_permanent_roles PASSED
test_temporary_permission_assignment_expires PASSED
test_rule_changes_result_compared_with_base_model PASSED
test_complete_mediation_every_call_logged PASSED
test_enforce_raises_and_denied_operation_is_not_executed PASSED
test_operation_log_contains_only_state_changes PASSED
test_policy_changes_go_to_operation_log PASSED
test_service_has_no_reference_to_policy PASSED
test_audit_reason_for_expired_right PASSED
test_a1_denial_spike_detected PASSED
test_a1_slow_denials_not_detected PASSED
test_a5_first_use_of_permission_detected PASSED
test_a5_ignores_baseline_and_denials PASSED
test_a5_object_granularity_is_more_sensitive PASSED
test_variant_detector_two_alerts_and_clean_log PASSED
test_who_has_access_changes_after_expiry PASSED
test_queries_are_dual_and_rbac_query_matches_full_scan PASSED
============================== 29 passed in 0.19s ==============================
```

ДОДАТОК В

Вивід demo.py

```
== 1. Таблиця очікуваних і фактичних рішень (з D2 / базова RBAC без D2)
10:00 alice   READ   hr01       очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль head (head читає кадрові)
10:00 alice   DELETE order02    очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль head (head видаляє наказ)
10:00 bob     WRITE  order03    очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль secretary (secretary)
10:00 bob     READ   hr01       очік. відмова факт. відмова база відмова
      явного дозволу немає (safe default) (немає права)
10:00 carol   READ   protocol01 очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль lecturer ← employee (успадкування employee)
10:00 carol   WRITE  grades01   очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль lecturer (сесія триває)
17:00 carol   WRITE  grades01   очік. відмова факт. відмова база дозвіл   ← D2
      тимчасове право неактивне — право lecturer на grades01: термін дії сплив 2026-10-26 16:00 (D2: сесія закінчилась)
10:00 dave    DELETE order02    очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль head (в. о. завідувача)
13:00 dave    DELETE order02    очік. відмова факт. відмова база дозвіл   ← D2
      тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00 (D2: в. о. закінчилось)
13:00 dave    WRITE  plan01     очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль lecturer (лишився lecturer)
10:00 frank   DELETE hr02       очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль hr (hr: R|W|D)
10:00 grace   READ   order01    очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль student (опублікований наказ)
10:00 grace   READ   order02    очік. відмова факт. відмова база відмова
      явного дозволу немає (safe default) (student)
10:00 mallory READ   grades02   очік. відмова факт. відмова база відмова
      явного дозволу немає (safe default) (student)
10:00 trent   READ   hr03       очік. дозвіл  факт. дозвіл  база дозвіл
      надано через роль auditor (auditor читає все)
10:00 trent   WRITE  hr03       очік. відмова факт. відмова база відмова
      явного дозволу немає (safe default) (auditor лише читає)
10:00 eve     READ   order01    очік. відмова факт. відмова база відмова
      явного дозволу немає (safe default) (невідомий суб'єкт)
Збіг очікуваних і фактичних: 17/17


== 2. Серія звернень через монітор (DocumentService → ReferenceMonitor)
Викликів монітора: 29, записів реєстраційного журналу: 29, з них відмов: 10
```

```
Операційний журнал: 108 записів (100 змін політики + 8 змін об'єктів)


Реєстраційний журнал (після базового періоду):
  10:05:00 carol   WRITE  grades01   дозвіл  надано через роль lecturer
  10:30:00 mallory READ   grades01   відмова явного дозволу немає (safe default)
  10:30:08 mallory READ   grades02   відмова явного дозволу немає (safe default)
  10:30:16 mallory READ   grades03   відмова явного дозволу немає (safe default)
  10:30:24 mallory READ   grades04   відмова явного дозволу немає (safe default)
  10:30:32 mallory READ   grades05   відмова явного дозволу немає (safe default)
  10:30:40 mallory READ   grades06   відмова явного дозволу немає (safe default)
  11:00:00 dave    DELETE order04    дозвіл  надано через роль head
  12:30:00 dave    DELETE order05    відмова тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00
  12:31:00 dave    DELETE order05    відмова тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00
  15:00:00 erin    WRITE  grades02   дозвіл  надано через роль lecturer
  16:30:00 erin    WRITE  grades02   відмова тимчасове право неактивне — право lecturer на grades02: термін дії сплив 2026-10-26 16:00
  17:00:00 trent   READ   order03    дозвіл  надано через роль auditor


Операційний журнал (зміни об'єктів):
  09:10 alice   write  grades01 довжина 22
  09:13 bob     write  order02 довжина 21
  09:20 bob     write  protocol01 довжина 24
  09:26 carol   write  plan01 довжина 20
  09:46 frank   write  hr02 довжина 18
  10:05 carol   write  grades01 довжина 22
  11:00 dave    delete order04
  15:00 erin    write  grades02 довжина 22


== 3. Детектори аномалій (A1 поріг 5 відмов / 60 с; A5 базовий період до 10:00)
  A1: mallory — 6 відмов за 60 с (від 2026-10-26 10:30)
  A5: dave уперше використав DELETE (order04, 2026-10-26 11:00)
  A5: erin уперше використав WRITE (grades02, 2026-10-26 15:00)
  A5 з granularity='object':
    A5: carol уперше використав WRITE/grades (grades01, 2026-10-26 10:05)
    A5: dave уперше використав DELETE/order (order04, 2026-10-26 11:00)
    A5: erin уперше використав WRITE/grades (grades02, 2026-10-26 15:00)
    A5: trent уперше використав READ/order (order03, 2026-10-26 17:00)
  «Чистий» журнал (16 записів): []


== 4. Запити «хто має доступ» і «до чого має доступ»
[2026-10-26 10:00]
  хто видаляє order05: ['alice', 'dave']
  хто пише grades03:  ['alice', 'carol', 'dave', 'erin']
  що видаляє dave:    ['order01', 'order02', 'order03', 'order04', 'order05']
  ролі dave:          ['employee', 'head', 'lecturer', 'secretary']
```

```
[2026-10-26 13:00]
  хто видаляє order05: ['alice']
  хто пише grades03:  ['alice', 'carol', 'dave', 'erin']
  що видаляє dave:    []
  ролі dave:          ['employee', 'lecturer']
  що читає carol: ['grades01', 'grades02', 'grades03', 'grades04', 'grades05', 'grades06', 'order01', 'order02', 'order03', 'order04',
'order05', 'plan01', 'plan02', 'plan03', 'plan04', 'protocol01', 'protocol02', 'protocol03', 'protocol04']
Узгодженість «s ∈ хто(o) ⇔ o ∈ що(s)»: 2070/2070 пар (10:00, 13:00, 17:00 × READ/WRITE/DELETE)


== 5. Тимчасове право спливає під час серії звернень (dave, DELETE order05)
  11:58 дозвіл  надано через роль head; хто видаляє order05: ['alice', 'dave']
  11:59 дозвіл  надано через роль head; хто видаляє order05: ['alice', 'dave']
  12:00 відмова тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00; хто видаляє order05: ['alice']
  12:01 відмова тимчасове право неактивне — призначення ролі head: термін дії сплив 2026-10-26 12:00; хто видаляє order05: ['alice']
```

ДОДАТОК Г

Лістинги mypolicy.py, population.py, mydetector.py

```
# mypolicy.py
"""ЛР2, варіант 14: RBAC з ієрархією ролей і тимчасовими правами (D2).


Модель (Sandhu et al., 1996; лекція 6):
  UA — призначення «суб'єкт → роль», PA — призначення «роль → права на об'єкт»,
  RH — ієрархія ролей: старша роль успадковує всі права молодших.
Додаткове правило D2: кожне призначення UA і PA може мати інтервал дії
[valid_from, valid_until). Поза інтервалом призначення не дає прав.


Рішення (формула 3.1 практикуму, deny-записів у варіанті немає):
  allow(s, o, a) = a ⊆ ⋃ { perm(r, o) : r ∈ RH*(active_roles(s, t)), PA(r, o) чинне в t }
Якщо дозволу немає — відмова (safe default).
"""
from __future__ import annotations


import dataclasses
import datetime as dt
import json
import time
from collections.abc import Callable, Iterable


from accesscontrol.monitor import AuditLog, Decision, Permission, Policy, ReferenceMonitor


Clock = Callable[[], float]
KYIV = dt.timezone(dt.timedelta(hours=2), "EET")   # синтетичні дані — лише для показу часу



def fmt_ts(ts: float) -> str:
    return dt.datetime.fromtimestamp(ts, KYIV).strftime("%Y-%m-%d %H:%M")



@dataclasses.dataclass(frozen=True)
```

```
class Interval:
    """Інтервал дії [start, end); None — без обмеження з цього боку."""

    start: float | None = None
    end: float | None = None

    def contains(self, t: float) -> bool:
        return (self.start is None or self.start <= t) and (self.end is None or t < self.end)

    def describe(self, t: float) -> str:
        if self.end is not None and t >= self.end:
            return f"термін дії сплив {fmt_ts(self.end)}"
        return f"діє лише з {fmt_ts(self.start)}"


ALWAYS = Interval()


@dataclasses.dataclass(frozen=True)
class Assignment:
    role: str
    interval: Interval = ALWAYS


@dataclasses.dataclass(frozen=True)
class Grant:
    perm: Permission
    interval: Interval = ALWAYS


def _via(assigned: str, role: str) -> str:
    """Як отримано право: «head» або «lecturer ← employee» (успадкування)."""
    return assigned if assigned == role else f"{assigned} ← {role}"


def _why(a: Assignment, role: str, obj: str, g: Grant, now: float) -> str:
    """Причина, з якої наявне право неактивне в момент now."""
    if not a.interval.contains(now):
```

```
        return f"призначення ролі {a.role}: {a.interval.describe(now)}"
    return f"право {role} на {obj}: {g.interval.describe(now)}"


class OperationLog:
    """Операційний журнал: лише виконані зміни стану (політики й об'єктів)."""

    def __init__(self, clock: Clock = time.time) -> None:
        self._clock = clock
        self._records: list[dict] = []

    def record(self, actor: str, operation: str, target: str, details: str = "") -> None:
        self._records.append({"ts": self._clock(), "actor": actor,
                              "operation": operation, "target": target, "details": details})

    def entries(self) -> list[dict]:
        return list(self._records)

    def dump(self) -> str:
        return "\n".join(json.dumps(r, ensure_ascii=False) for r in self._records)


class ClockedAuditLog(AuditLog):
    """Реєстраційний журнал каркаса з тим самим годинником, що й політика."""

    def __init__(self, clock: Clock = time.time) -> None:
        super().__init__()
        self._clock = clock

    def _now(self) -> float:
        return self._clock()


class VariantPolicy(Policy):
    """RBAC з ієрархією ролей і тимчасовими призначеннями."""

    def __init__(self, clock: Clock = time.time, oplog: OperationLog | None = None) -> None:
        self._clock = clock
```

```
    self._oplog = oplog or OperationLog(clock)
    self._roles: set[str] = set()
    self._juniors: dict[str, set[str]] = {}              # RH: старша → безпосередні молодші
    self._ua: dict[str, list[Assignment]] = {}           # UA: суб'єкт → призначені ролі
    self._pa: dict[tuple[str, str], list[Grant]] = {}    # PA: (роль, об'єкт) → права

# ---------- адміністрування політики (кожна зміна — в операційний журнал) ----------

def add_role(self, role: str, inherits: Iterable[str] = ()) -> None:
    self._roles.add(role)
    self._juniors.setdefault(role, set())
    self._oplog.record("admin", "add_role", role)
    for junior in inherits:
        self.add_inheritance(role, junior)

def add_inheritance(self, senior: str, junior: str) -> None:
    self._require_role(senior)
    self._require_role(junior)
    if senior in self._closure(junior):
        raise ValueError(f"цикл в ієрархії ролей: {junior} уже старша за {senior}")
    self._juniors[senior].add(junior)
    self._oplog.record("admin", "add_inheritance", senior, f"успадковує {junior}")

def assign(self, subject: str, role: str, valid_from: float | None = None,
           valid_until: float | None = None) -> None:
    self._require_role(role)
    interval = Interval(valid_from, valid_until)
    self._ua.setdefault(subject, []).append(Assignment(role, interval))
    self._oplog.record("admin", "assign", subject, f"роль {role}{self._period(interval)}")

def revoke(self, subject: str, role: str) -> None:
    self._ua[subject] = [a for a in self._ua.get(subject, []) if a.role != role]
    self._oplog.record("admin", "revoke", subject, f"роль {role}")

def permit(self, role: str, obj: str, perm: Permission, valid_from: float | None = None,
           valid_until: float | None = None) -> None:
    self._require_role(role)
    interval = Interval(valid_from, valid_until)
    self._pa.setdefault((role, obj), []).append(Grant(perm, interval))
    self._oplog.record("admin", "permit", obj, f"{role}: {perm.name}{self._period(interval)}")
```

```
# ---------- рішення (довірена частина: evaluate і _closure) ----------

def evaluate(self, subject: str, obj: str, action: Permission) -> Decision:
    now = self._clock()
    effective = Permission.NONE   # права, чинні зараз
    inactive = Permission.NONE    # права поза інтервалом дії (D2)
    why_inactive, sources = "", []
    for a in self._ua.get(subject, []):
        for role in sorted(self._closure(a.role)):
            for g in self._pa.get((role, obj), []):
                if not g.perm & action:
                    continue
                if a.interval.contains(now) and g.interval.contains(now):
                    effective |= g.perm
                    sources.append(_via(a.role, role))
                else:
                    inactive |= g.perm
                    why_inactive = why_inactive or _why(a, role, obj, g, now)
    if action & effective == action:
        via = ", ".join(dict.fromkeys(sources))
        return Decision(True, "надано через роль " + via)
    if action & (effective | inactive) == action:
        return Decision(False, "тимчасове право неактивне — " + why_inactive)
    return Decision(False, "явного дозволу немає (safe default)")


def _closure(self, role: str) -> set[str]:
    """Роль разом з усіма молодшими (транзитивне замикання RH)."""
    seen, stack = set(), [role]
    while stack:
        r = stack.pop()
        if r not in seen:
            seen.add(r)
            stack.extend(self._juniors.get(r, ()))
    return seen


# ---------- запити, природні для RBAC ----------


def active_roles(self, subject: str) -> set[str]:
    """Ролі, чинні зараз, з урахуванням успадкування."""
    now = self._clock()
    roles: set[str] = set()
    for a in self._ua.get(subject, []):
```

```
            if a.interval.contains(now):
                roles |= self._closure(a.role)
        return roles

    def accessible_objects(self, subject: str, action: Permission) -> list[str]:
        """«До чого має доступ суб'єкт»: суб'єкт → ролі → PA, без обходу всіх об'єктів."""
        roles = self.active_roles(subject)
        candidates = {obj for (role, obj) in self._pa if role in roles}
        return sorted(o for o in candidates if self.evaluate(subject, o, action).allowed)

    @property
    def subjects(self) -> list[str]:
        return sorted(self._ua)

    @staticmethod
    def _period(interval: Interval) -> str:
        if interval == ALWAYS:
            return ""
        start = fmt_ts(interval.start) if interval.start is not None else "…"
        end = fmt_ts(interval.end) if interval.end is not None else "…"
        return f" [{start} – {end})"

    def _require_role(self, role: str) -> None:
        if role not in self._roles:
            raise KeyError(f"невідома роль {role}")



def objects_accessible_by(policy: Policy, subject: str, objects: Iterable[str],
                          action: Permission) -> list[str]:
    """«До чого має доступ суб'єкт» повним обходом об'єктів (двоїсте subjects_with_access)."""
    return [o for o in objects if policy.evaluate(subject, o, action).allowed]



class DocumentService:
    """Підсистема зберігання документів. Має лише монітор — не політику.

    Кожна операція спершу проходить ReferenceMonitor.enforce (повне посередництво);
    у операційний журнал потрапляють лише виконані зміни (WRITE, DELETE).
    """
```

```
    def __init__(self, monitor: ReferenceMonitor, oplog: OperationLog,
                 documents: dict[str, str]) -> None:
        self._monitor = monitor
        self._oplog = oplog
        self._docs = dict(documents)

    def read(self, subject: str, obj: str) -> str:
        self._monitor.enforce(subject, obj, Permission.READ)
        return self._docs[obj]

    def write(self, subject: str, obj: str, text: str) -> None:
        self._monitor.enforce(subject, obj, Permission.WRITE)
        self._docs[obj] = text
        self._oplog.record(subject, "write", obj, f"довжина {len(text)}")

    def delete(self, subject: str, obj: str) -> None:
        self._monitor.enforce(subject, obj, Permission.DELETE)
        del self._docs[obj]
        self._oplog.record(subject, "delete", obj)
```

```
# population.py
"""Наповнення політики варіанта 14: 10 суб'єктів, 23 об'єкти (усі дані вигадані).

Предметна область — СЕД кафедри з ЛР1. Ієрархія ролей (старша ≻ молодша):

    head ≻ lecturer ≻ employee
    head ≻ secretary ≻ employee
    hr ≻ employee
    student, auditor — окремі кореневі ролі

Тимчасові права (D2):
  - dave — в. о. завідувача (роль head) лише 09:00–12:00 26.10.2026;
  - роль lecturer має WRITE на відомості лише під час сесії, до 16:00 26.10.2026.
"""
from __future__ import annotations

import datetime as dt

from accesscontrol.monitor import Permission
```

```
from mypolicy import KYIV, Clock, OperationLog, VariantPolicy

R, W, D = Permission.READ, Permission.WRITE, Permission.DELETE



def at(hour: int, minute: int = 0, second: int = 0, day: int = 26) -> float:
    """Синтетичний час 2026-10-<day> hh:mm:ss (Київ) як мітка часу."""
    return dt.datetime(2026, 10, day, hour, minute, second, tzinfo=KYIV).timestamp()



SUBJECTS = ["alice", "bob", "carol", "dave", "erin", "frank", "grace", "heidi", "mallory", "trent"]

ORDERS = [f"order{i:02d}" for i in range(1, 6)]        # накази щодо здобувачів
GRADES = [f"grades{i:02d}" for i in range(1, 7)]       # екзаменаційні відомості
PROTOCOLS = [f"protocol{i:02d}" for i in range(1, 5)]  # протоколи засідань кафедри
PLANS = [f"plan{i:02d}" for i in range(1, 5)]          # індивідуальні плани викладачів
HR = [f"hr{i:02d}" for i in range(1, 5)]               # кадрові документи
OBJECTS = ORDERS + GRADES + PROTOCOLS + PLANS + HR

ACTING_HEAD = (at(9), at(12))       # dave: в. о. завідувача
SESSION_END = at(16)                # кінець сесії: lecturer більше не редагує відомості



def build_policy(clock: Clock, oplog: OperationLog | None = None,
                 temporal: bool = True) -> VariantPolicy:
    """temporal=False — та сама політика без правила D2 (для порівняння з базовою моделлю)."""
    p = VariantPolicy(clock, oplog)
    for role in ("employee", "student", "auditor"):
        p.add_role(role)
    p.add_role("lecturer", inherits=["employee"])
    p.add_role("secretary", inherits=["employee"])
    p.add_role("hr", inherits=["employee"])
    p.add_role("head", inherits=["lecturer", "secretary"])


    def period(start, end):
        return (start, end) if temporal else (None, None)


    for o in ORDERS + PROTOCOLS:
        p.permit("employee", o, R)
```

```
    for o in ORDERS + PROTOCOLS:
        p.permit("secretary", o, W)
    for o in PLANS:
        p.permit("lecturer", o, R | W)
    for o in GRADES:
        p.permit("lecturer", o, R)
        p.permit("lecturer", o, W, *period(None, SESSION_END))
        p.permit("head", o, W)
    for o in ORDERS:
        p.permit("head", o, D)
    for o in HR:
        p.permit("head", o, R)
        p.permit("hr", o, R | W | D)
    p.permit("student", "order01", R)     # опублікований наказ
    for o in OBJECTS:
        p.permit("auditor", o, R)


    p.assign("alice", "head")
    p.assign("bob", "secretary")
    p.assign("carol", "lecturer")
    p.assign("dave", "lecturer")
    p.assign("dave", "head", *period(*ACTING_HEAD))
    p.assign("erin", "lecturer")
    p.assign("frank", "hr")
    for s in ("grace", "heidi", "mallory"):
        p.assign(s, "student")
    p.assign("trent", "auditor")
    return p



class FakeClock:
    """Керований годинник для демонстрації та тестів."""

    def __init__(self, t: float) -> None:
        self.t = t

    def __call__(self) -> float:
        return self.t

    def set(self, t: float) -> None:
        self.t = t
```

```
# mydetector.py
"""Детектори аномалій варіанта 14 (лекція 7).


A1 — сплеск відмов у доступі від одного суб'єкта;
A5 — використання прав, не задіяних раніше.
Пороги передаються параметрами, як у FanInDetector навчального прикладу.
"""
from __future__ import annotations


import collections
import re


from accesscontrol.monitor import AnomalyDetector, AuditLog
from mypolicy import fmt_ts



class DenialSpikeDetector(AnomalyDetector):
    """A1: не менше `threshold` відмов одному суб'єкту за `window` секунд."""


    def __init__(self, threshold: int = 5, window: float = 60.0) -> None:
        self.threshold = threshold
        self.window = window


    def scan(self, log: AuditLog) -> list[str]:
        denials: dict[str, list[float]] = collections.defaultdict(list)
        for r in log.entries():
            if not r["allowed"]:
                denials[r["subject"]].append(r["ts"])
        alerts = []
        for subject, times in sorted(denials.items()):
            times.sort()
            best, best_start, left = 0, 0.0, 0
            for right, t in enumerate(times):          # ковзне вікно двома вказівниками
                while t - times[left] > self.window:
                    left += 1
                if right - left + 1 > best:
                    best, best_start = right - left + 1, times[left]
            if best >= self.threshold:
                alerts.append(f"A1: {subject} — {best} відмов за {self.window:.0f} с "
                              f"(від {fmt_ts(best_start)})")
        return alerts
```

```
def object_group(obj: str) -> str:
    """grades03 → grades: тип документа без номера."""
    return re.sub(r"\d+$", "", obj)



class NewPermissionDetector(AnomalyDetector):
    """A5: дозволене звернення з правом, якого суб'єкт не використовував у базовому періоді.

    Базовий період — записи з ts < baseline_until (навчання профілю).
    granularity="action" — профіль «суб'єкт → дія»;
    granularity="object" — «суб'єкт → (дія, тип документа)», чутливіший.
    """

    def __init__(self, baseline_until: float, granularity: str = "action") -> None:
        if granularity not in ("action", "object"):
            raise ValueError("granularity: 'action' або 'object'")
        self.baseline_until = baseline_until
        self.granularity = granularity

    def _key(self, r: dict) -> tuple[str, ...]:
        if self.granularity == "action":
            return (r["action"],)
        return (r["action"], object_group(r["object"]))

    def scan(self, log: AuditLog) -> list[str]:
        used: dict[str, set[tuple[str, ...]]] = collections.defaultdict(set)
        alerts = []
        for r in sorted(log.entries(), key=lambda e: e["ts"]):
            if not r["allowed"]:
                continue
            key = self._key(r)
            if r["ts"] >= self.baseline_until and key not in used[r["subject"]]:
                alerts.append(f"A5: {r['subject']} уперше використав {'/'.join(key)} "
                              f"({r['object']}, {fmt_ts(r['ts'])})")
            used[r["subject"]].add(key)
        return alerts
```

```
class VariantDetector(AnomalyDetector):
    """Обидві аномалії варіанта в одному скануванні."""

    def __init__(self, baseline_until: float, threshold: int = 5, window: float = 60.0,
                 granularity: str = "action") -> None:
        self.detectors = [DenialSpikeDetector(threshold, window),
                          NewPermissionDetector(baseline_until, granularity)]

    def scan(self, log: AuditLog) -> list[str]:
        return [alert for d in self.detectors for alert in d.scan(log)]
```